

Actividad scripting

Pedro Gutiérrez La Ossa 1 ASIR

Índice

1. Primer script
2. Segundo script
3. Tercer script
4. Cuarto script
5. Quinto script
6. Sexto script
7. séptimo script
8. octavo script
9. noveno script
10. décimo script
11. decimoprimer script
12. decimosegundo script

1. El script usa `#!/bin/bash` para definir el intérprete. Con `mkdir -p` `filedir` crea la carpeta (evitando errores si ya existe). Un bucle `for i in $(seq 1 10)` recorre del 1 al 10 y con `touch "filedir/file$i"` crea los archivos `file1` al `file10`. Finalmente, `ls -l filedir/` muestra el listado detallado.

```
GNU nano 8.4 script1.sh *
#!/bin/bash

mkdir -p filedir

for i in $(seq 1 10)
do
    touch "filedir/file$i"
done

ls -l filedir/
```

```
pedro@pedro:~$ ./script1.sh
total 0
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file1
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file10
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file2
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file3
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file4
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file5
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file6
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file7
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file8
-rw-rw-r-- 1 pedro pedro 0 Apr 29 17:31 file9
```

2. El script define el intérprete con `#!/bin/bash`. Genera un número aleatorio entre 0 y 100 con `$((RANDOM % 101))` e inicializa `intentos=0` y `acertado=false`. Muestra un mensaje de bienvenida y entra en un bucle `while` que se repite hasta acertar. Dentro del bucle pide un número con `read -p`. Usa `if` con `-lt` para comprobar si es menor (muestra "El número que buscas es mayor"), `elif` con `-gt` para si es mayor (muestra "El número que buscas es menor"), y `else` para cuando acierta: muestra la enhorabuena y el número de intentos, y cambia `acertado=true` para salir del bucle. En cada

intento incrementa el contador.

```
GNU nano 8.4                                script2.sh *
#!/bin/bash

numero_escondido=$((RANDOM % 101))
intentos=0
acertado=false

echo "=== ADIVINA EL NÚMERO ESCONDIDO (0-100) ==="

while [ "$acertado" = false ]
do
    read -p "Introduce un número: " numero_usuario
    if [ "$numero_usuario" -lt "$numero_escondido" ]
    then
        echo "El número que buscas es mayor. Sigue intentándolo"
        intentos=$((intentos + 1))
    elif [ "$numero_usuario" -gt "$numero_escondido" ]
    then
        echo "El número que buscas es menor. Sigue intentándolo"
        intentos=$((intentos + 1))
    else
        intentos=$((intentos + 1))
        echo "Enhorabuena !. Has acertado el número escondido."
        echo "El número de intentos ha sido $intentos"
        acertado=true
    fi
done
```

```
pedro@pedro:~$ ./script2.sh
=== ADIVINA EL NÚMERO ESCONDIDO (0-100) ===
Introduce un número: 50
El número que buscas es mayor. Sigue intentándolo
Introduce un número: 70
El número que buscas es mayor. Sigue intentándolo
Introduce un número: 80
El número que buscas es menor. Sigue intentándolo
Introduce un número: 74
El número que buscas es mayor. Sigue intentándolo
Introduce un número: 76
El número que buscas es mayor. Sigue intentándolo
Introduce un número: 78
El número que buscas es mayor. Sigue intentándolo
Introduce un número: 79
Enhorabuena !. Has acertado el número escondido.
El número de intentos ha sido 7
```

3. El script define el intérprete con `#!/bin/bash`. Con `read -p` pide un nombre de usuario y lo guarda. Usa `if id "$usuario" &>/dev/null` para verificar si existe, donde `&>/dev/null` oculta la salida y solo importa el código de retorno. Si existe (código 0), muestra: `$(whoami)` para el usuario que ejecuta el script, `$(groups $usuario)` para los grupos del usuario buscado, y `grep "^$usuario:"`

/etc/passwd | cut -d: -f7 para obtener su terminal asociado. Si no existe, el else muestra que no está creado.

```
GNU nano 8.4                                script3.sh *
#!/bin/bash

read -p "Introduce el nombre de usuario a buscar: " usuario

if id "$usuario" &>/dev/null
then
    echo "El usuario $usuario está creado en el sistema y estos son sus datos:"
    echo ""
    echo "- Nombre del usuario con el que ha hecho login: $(whoami)"
    echo "- Grupos a los que pertenece: $(groups $usuario)"
    echo "- Terminal de comandos asociado: $(grep "^$usuario:" /etc/passwd | cut -d: -f7)"
else
    echo "El usuario $usuario no está creado en el sistema"
fi
```

```
pedro@pedro:~$ ./script3.sh
Introduce el nombre de usuario a buscar: Pedro
El usuario Pedro no está creado en el sistema
```

4. El script usa `#!/bin/bash`. Con dos `read -p` pide dos números. Muestra un menú con `echo` y pide la operación. Usa `case` para ejecutar según la operación elegida: suma (`$((num1 + num2))`), resta (`$((num1 - num2))`), multiplicación (escapa `\*` y usa `$((num1 * num2))`), división (comprueba división entre cero con `if [ "$num2" -eq 0 ]`), potencia (`$((num1 ** $num2))`). El comodín `*` captura errores. Cada bloque termina con `;;` y el `case` con `esac`. Finalmente muestra el resultado con el formato solicitado.

```
GNU nano 8.4                                script4.sh *
#!/bin/bash

read -p "Introduce el primer número: " num1
read -p "Introduce el segundo número: " num2

echo "Operaciones disponibles:"
echo " + : Suma"
echo " - : Resta"
echo " * : Multiplicación"
echo " / : División"
echo " ^ : Potencia"
read -p "Elige la operación: " operacion

case $operacion in
+)
    resultado=$((num1 + num2))
    echo "El resultado de '$num1' '$operacion' '$num2' es '$resultado'"
    ;;
-)
    resultado=$((num1 - num2))
    echo "El resultado de '$num1' '$operacion' '$num2' es '$resultado'"
    ;;
\*)
    resultado=$((num1 * num2))
    echo "El resultado de '$num1' '$operacion' '$num2' es '$resultado'"
    ;;
/)
    if [ "$num2" -eq 0 ]
    then
        echo "Error: No se puede dividir entre cero"
    else
        resultado=$((num1 / num2))
        echo "El resultado de '$num1' '$operacion' '$num2' es '$resultado'"
    fi
    ;;
^)
    resultado=$((num1 ** num2))
    echo "El resultado de '$num1' '$operacion' '$num2' es '$resultado'"
    ;;
*)
    echo "Error: Operación no válida"
    ;;
esac
```

```
pedro@pedro:~$ ./script4.sh
Introduce el primer número: 3
Introduce el segundo número: 4
Operaciones disponibles:
+ : Suma
- : Resta
* : Multiplicación
/ : División
^ : Potencia
Elige la operación: +
El resultado de '3' '+' '4' es '7'
```

5. He puesto `#!/bin/bash`. Con `hora=$(date +%H)` obtengo la hora actual en formato 24 horas. Con `if [ $hora -ge 7 ] && [ $hora -lt 14 ]` compruebo si es de 7:00 a 13:59 y muestro "Buenos días". Con `elif [ $hora -ge 14 ] && [ $hora -lt 21 ]` compruebo si es de 14:00 a 20:59 y muestro "Buenas tardes". En cualquier otro caso muestro "Buenas noches".

```
GNU nano 8.4 script5.sh
#!/bin/bash

hora=$(date +%H)

if [ $hora -ge 7 ] && [ $hora -lt 14 ]
then
    echo "Buenos días"
elif [ $hora -ge 14 ] && [ $hora -lt 21 ]
then
    echo "Buenas tardes"
else
    echo "Buenas noches"
fi
```

```
pedro@pedro:~$ ./script5.sh
Buenas tardes
```

6.He puesto `#!/bin/bash`. Para for uso `seq 1 1000` para generar números del 1 al 1000 y sumarlos. Para while uso `[ $contador -le 1000 ]` que significa "mientras sea menor o igual a 1000". Para until uso `[ $contador -gt 1000 ]` que significa "hasta que sea mayor que 1000". Los tres bucles suman del 1 al 1000.

```
pedro@pedro: ~ — nano script6.sh
GNU nano 8.4 script6.sh *
#!/bin/bash

suma_for=0
for i in $(seq 1 1000)
do
    suma_for=$((suma_for + i))
done
echo "Suma con FOR: $suma_for"

suma_while=0
contador=1
while [ $contador -le 1000 ]
do
    suma_while=$((suma_while + contador))
    contador=$((contador + 1))
done
echo "Suma con WHILE: $suma_while"

suma_until=0
contador=1
until [ $contador -gt 1000 ]
do
    suma_until=$((suma_until + contador))
    contador=$((contador + 1))
done
echo "Suma con UNTIL: $suma_until"
```

```
pedro@pedro:~$ ./script6.sh
Suma con FOR: 500500
Suma con WHILE: 500500
Suma con UNTIL: 500500
```

7.He puesto `#!/bin/bash`. Con `[ $# -ne 1 ]` compruebo que se pase un solo parámetro. Si no, muestro error y salgo con exit 1. Con `$((numero % 2))` calculo el resto de dividir entre 2. Si es 0 es par, si no es impar.

```
GNU nano 8.4                                script7.sh *
#!/bin/bash

if [ $# -ne 1 ]
then
    echo "Error: Debes pasar un número como parámetro"
    echo "Ejemplo: ./script3.sh 5"
    exit 1
fi

numero=$1

if [ $((numero % 2)) -eq 0 ]
then
    echo "El número $numero es PAR"
else
    echo "El número $numero es IMPAR"
fi

pedro@pedro:~$ ./script7.sh
Error: Debes pasar un número como parámetro
Ejemplo: ./script3.sh 5
```

8.He puesto `#!/bin/bash`. Con `for num in 7 8 9` recorro los números. Dentro, otro `for i in $(seq 1 10)` recorre del 1 al 10 y multiplica `$num` por `$i` mostrando el resultado.

```
pedro@pedro:~$ nano script8.sh
GNU nano 8.4 script8.sh
#!/bin/bash

for num in 7 8 9
do
    echo "=== Tabla de multiplicar del $num ==="
    for i in $(seq 1 10)
    do
        resultado=$((num * i))
        echo "$num x $i = $resultado"
    done
done
echo ""
```

```
pedro@pedro:~$ ./script8.sh
=== Tabla de multiplicar del 7 ===
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

=== Tabla de multiplicar del 8 ===
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
8 x 10 = 80

=== Tabla de multiplicar del 9 ===
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
```

9.He puesto `#!/bin/bash`. Con `read -p` pido la palabra. Con `tr` convierto a minúsculas. `${#palabra}` da la longitud. Con `for` recorro cada letra y extraigo una con `${palabra:$i:1}`. Con `case` compruebo si es vocal y sumo.

```
pedro@pedro: ~ — nano script9.sh
GNU nano 8.4 script9.sh *
#!/bin/bash

read -p "Introduce una palabra: " palabra

palabra=$(echo $palabra | tr '[:upper:]' '[:lower:]')

vocales=0

for ((i=0; i<${#palabra}; i++))
do
    letra="${palabra:$i:1}"
    case $letra in
        a|e|i|o|u)
            vocales=$((vocales + 1))
            ;;
    esac
done

echo "La palabra '$palabra' tiene $vocales vocales"
```

```
pedro@pedro:~$ ./script9.sh
Introduce una palabra: hola
La palabra 'hola' tiene 2 vocales
```

10.He puesto `#!/bin/bash`. Pido dos números y la operación. Con `case` ejecuto la operación elegida. Para la multiplicación uso `\*` para escapar

el asterisco. Para división compruebo que no sea cero.

```
GNU nano 8.4                                script10.sh
#!/bin/bash

read -p "Introduce el primer número: " num1
read -p "Introduce el segundo número: " num2

echo "Operaciones disponibles:"
echo " + : Suma"
echo " - : Resta"
echo " * : Multiplicación"
echo " / : División"
read -p "Elige la operación: " operacion

case $operacion in
+)
    resultado=$((num1 + num2))
    echo "$num1 + $num2 = $resultado"
    ;;
-)
    resultado=$((num1 - num2))
    echo "$num1 - $num2 = $resultado"
    ;;
\*)
    resultado=$((num1 * num2))
    echo "$num1 x $num2 = $resultado"
    ;;
/)
    if [ "$num2" -eq 0 ]
    then
        echo "Error: No se puede dividir entre cero"
    else
        resultado=$((num1 / num2))
        echo "$num1 / $num2 = $resultado"
    fi
    ;;
*)
    echo "Error: Operación no válida"
    ;;
```

```
pedro@pedro:~$ ./script10.sh
Introduce el primer número: 17
Introduce el segundo número: 31
Operaciones disponibles:
+ : Suma
- : Resta
* : Multiplicación
/ : División
Elige la operación: *
```

11. He puesto `#!/bin/bash`. Defino dos funciones. En `obtener_info_red()` uso `ip addr show` y filtros para obtener IP, máscara, red y broadcast. En `escanear_ips()` recorro todas las IPs de la red (1 a 254) y hago ping a cada una. Si responde está ocupada, si no está libre. Llamo a las dos funciones.

```

GNU nano 8.4                                script11.sh *
#!/bin/bash

obtener_info_red() {
    echo "=== INFORMACIÓN DE RED ==="
    ip addr show | grep -E "inet " | grep -v "127.0.0.1" | head -1 | while read linea
    do
        ip_cidr=$(echo $linea | awk '{print $2}')
        echo "IP con CIDR: $ip_cidr"

        ip=$(echo $ip_cidr | cut -d/ -f1)
        mascara=$(echo $ip_cidr | cut -d/ -f2)
        echo "Dirección IP: $ip"
        echo "Máscara de subred: /$mascara"

        IFS=. read -r a b c d <<< "$ip"
        red="$a.$b.$c.0"
        broadcast="$a.$b.$c.255"
        echo "Red: $red"
        echo "Broadcast: $broadcast"
        echo "Tipo de red: /$mascara (CIDR)"
    done
}

escanear_ips() {
    echo ""
    echo "=== ESCANEEO DE IPS ==="

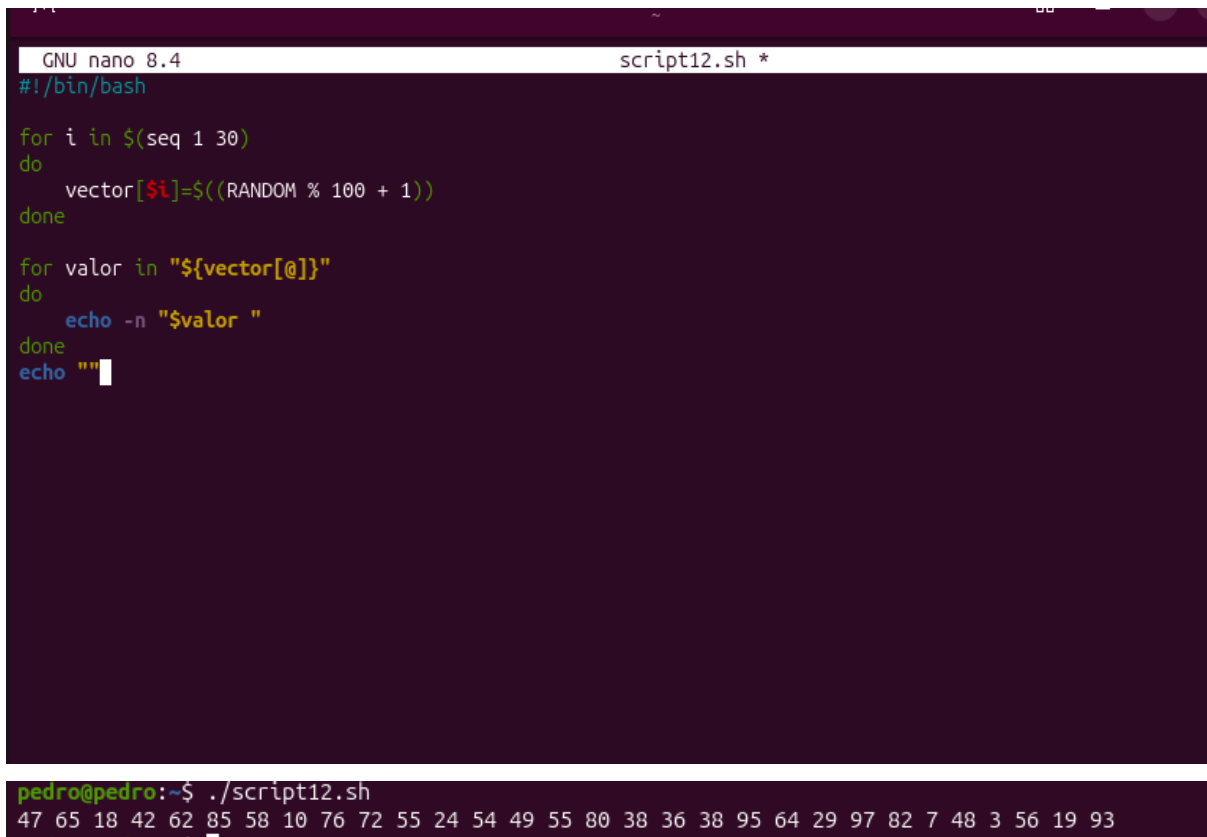
    ip_actual=$(ip addr show | grep -E "inet " | grep -v "127.0.0.1" | head -1 | awk '{print $2}' | cut -d/ -f1)
    IFS=. read -r a b c d <<< "$ip_actual"

    for i in $(seq 1 254)
    do
        ip="$a.$b.$c.$i"
        if ping -c 1 -W 1 $ip &>/dev/null
        then
            echo "IP $ip: OCUPADA"
        else
            echo "IP $ip: LIBRE"
        fi
    done
}

```

12. He puesto `#!/bin/bash`. Con `for i in $(seq 1 30)` recorro 30 veces y asigno a cada posición del vector un número aleatorio entre 1 y 100 con

$\$((RANDOM \% 100 + 1))$. Luego con `for valor in "${vector[@]}"` recorro y muestro con `echo -n` para que salga en una sola línea.



```
GNU nano 8.4 script12.sh *
#!/bin/bash

for i in $(seq 1 30)
do
    vector[$i]=$((RANDOM % 100 + 1))
done

for valor in "${vector[@]}"
do
    echo -n "$valor "
done
echo ""
```

pedro@pedro:~\$./script12.sh
47 65 18 42 62 85 58 10 76 72 55 24 54 49 55 80 38 36 38 95 64 29 97 82 7 48 3 56 19 93

13. He puesto `#!/bin/bash`. Pido 10 números y los guardo. Para orden inverso recorro de atrás adelante con `for ((i=10; i>=1; i--))`. Para orden de mayor a menor uso `printf` y `sort -nr`. Para la suma recorro y acumulo.

La media es la suma dividida entre 10.

```
pedro@pedro: ~ — nano script13.sh
GNU nano 8.4 script13.sh *
#!/bin/bash

for i in $(seq 1 10)
do
    read -p "Introduce el número $i: " vector[$i]
done

echo ""
echo "=== Orden inverso ==="
for ((i=10; i>=1; i--))
do
    echo -n "${vector[$i]} "
done

echo ""
echo "=== Orden de mayor a menor ==="
ordenado=$(printf "%s\n" "${vector[@]}" | sort -nr)
for valor in "${ordenado[@]}"
do
    echo -n "$valor "
done

suma=0
for valor in "${vector[@]}"
do
    suma=$((suma + valor))
done
echo ""
echo "=== Suma total ==="
echo "Suma: $suma"

media=$((suma / 10))
echo "=== Media aritmética ==="
echo "Media: $media"
```

```
pedro@pedro:~$ ./script13.sh
```

```
Introduce el número 1: 8
```

```
Introduce el número 2: 5
```

```
Introduce el número 3: 7
```

```
Introduce el número 4: 8
```

```
Introduce el número 5: 9
```

```
Introduce el número 6: 0
```

```
Introduce el número 7: 1
```

```
Introduce el número 8: 2
```

```
Introduce el número 9: 4
```

```
Introduce el número 10: 6
```

```
=== Orden inverso ===
```

```
6 4 2 1 0 9 8 7 5 8
```

```
=== Orden de mayor a menor ===
```

```
9 8 8 7 6 5 4 2 1 0
```

```
=== Suma total ===
```

```
Suma: 50
```

```
=== Media aritmética ===
```

```
Media: 5
```

Conclusión

He usado `chmod +x` para que se pueda ejecutar los scripts de la tarea. Además con scripting podemos automatizar un montón de tareas.